Plan 3 — V-Formation

Formation mission · domain `v-formation-drone-mission` · plan kind `formation`
A complete walkthrough of domain.pddl, problem.pddl and every Python function that runs them.

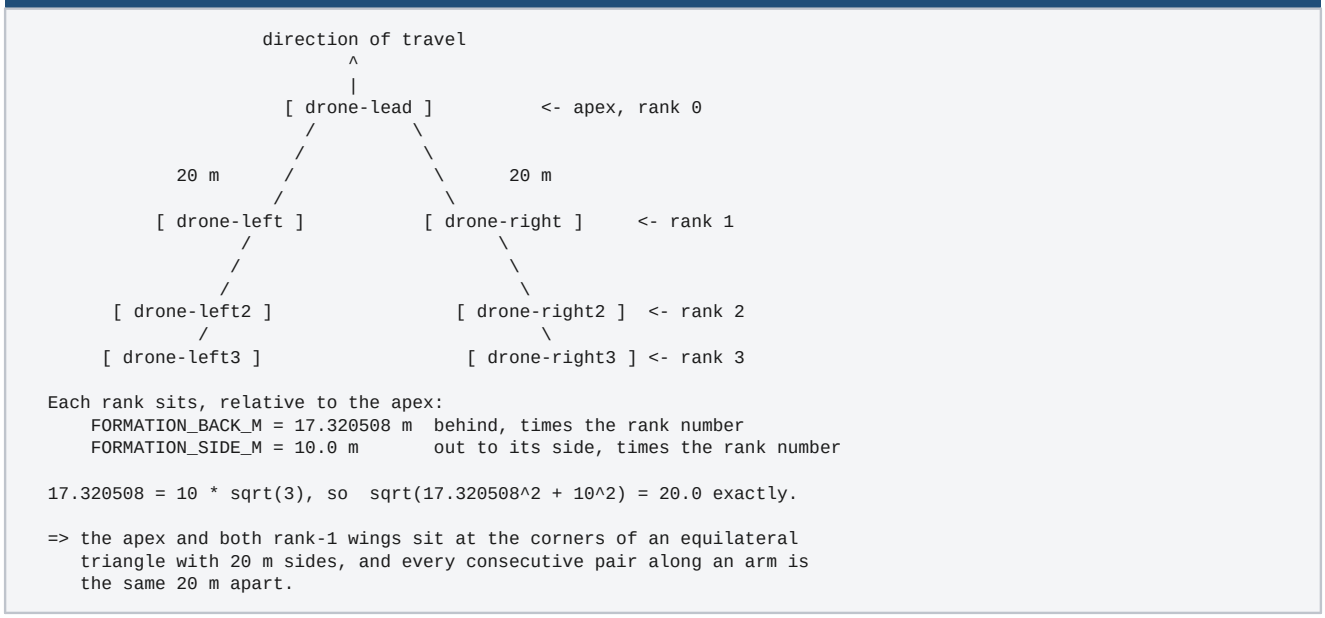
1. What this plan does

V-Formation is picked exactly like Travell - a **source**, a **destination**, and optionally a **restricted area** - but the swarm flies as a rigid V rather than as independent aircraft sharing a route. One drone is the **apex** (also called the leader); the rest are split evenly between a left wing and a right wing, each wing a straight arm running back from the apex.

The defining idea of this plan, and the thing that distinguishes it from the first two, is **the planner only ever plans one route**. ENHSP solves the apex's corridor alone. Every wing drone's track is then derived **geometrically** from the apex's flown path - held parallel to it, a fixed distance back and out to the side. The formation's shape is not something the planner discovers; it is imposed afterwards.

1.1 The V's geometry

The V, and where the two magic numbers come from



The drone count must be **odd and at least 3**: one apex plus a matched pair per rank. Five drones gives two ranks per wing, seven gives three, and so on.

1.2 The pipeline, end to end

The V-Formation planning pipeline

```
GUI: operator picks source + destination (+ optional no-fly polygon)
     and ticks N drones (N odd, N >= 3)
                  |
                  v
services/plan_service.py   run_async(...) -> _Worker.run(...) -> _run(...)
     plan_kind() == "formation"  ->  _run_formation(...)
                  |
                  +--> expand_formation_drones(template, N)
                  |       rank-1 wings -> ranks 1..(N-1)/2, both sides
                  |       plus a rebuilt wing-neighbor chain
                  |
                  +--> retarget_formation_problem(..., no_fly_buffer_m=
                  |         NO_FLY_BUFFER_M + FORMATION_SIDE_M * ranks)
                  |       move the graph; bend around the zone; recompute
                  |       every wing's slot-direction from the new bearing
                  v
     plans/vformation/runs/<timestamp>/problem.pddl
                  |
                  v
engine/pddl_planner.py  run_enhsp(...)
     -> extract_formation_corridor(steps)
        the apex's (formation-cruise ...) legs  ->  ONE corridor
                  |
                  v
formation_wing_routes(lead_route, back_m=BACK*r, side_m=SIDE*r)
     called once per rank  ->  a left and a right track per rank
                  |
                  v
PlanRunResult(per_drone_waypoints = {lead, left, right, left2, right2, ...})
```

2. domain.pddl explained

2.1 Header, requirements and types

domain.pddl - lines 1-22

```
(define (domain v-formation-drone-mission)

    (:requirements
        :strips :typing :equality :numeric-fluents
        :negative-preconditions :disjunctive-preconditions
        :universal-preconditions :conditional-effects)

    (:types
        drone
        location
        controller
        direction
    ))
```

The requirements list is noticeably longer than Travell's, and two additions carry real weight.

`:universal-preconditions` enables the `(forall (?w - drone) ...)` constructs that `establish-v-formation`, `close-up-wings` and `complete-formation-mission` all use. This is what lets a single action reason about **every wing drone at once**, without naming them - and it is the reason the domain never has to change when the drone count changes.

`:conditional-effects` enables the `(when ...)` clauses that let `close-up-wings` move all the wings in one action.

The `direction` type is new. It exists for the eight compass-point objects the problem file declares, used by the purely descriptive `slot-direction` predicate.

2.2 Predicates

Group A - position, and the formation's own position

```
;; Drone position
(at ?d - drone ?l - location)

;; Current location of the whole formation
(formation-at ?l - location)

;; Mission locations and routes
(source ?l - location) (destination ?l - location)
(connected ?from - location ?to - location)
(safe-route ?from - location ?to - location)

;; Flight states
(landed ?d - drone) (airborne ?d - drone)
(flying ?d - drone) (at-cruise-altitude ?d - drone)
```

`(formation-at ?l)` is the new idea, and the key to the whole domain. The formation as a **whole** has a position, distinct from any individual drone's. `formation-cruise` moves both the apex and this formation marker; `close-up-wings` then brings the wings to wherever the marker now is. That split is what produces the leader-moves-then-wings-follow rhythm.

The flight states are a genuine progression Travell lacks: `landed`, then `airborne` after takeoff, then `at-cruise-altitude` after the climb. A drone must pass through all three before it can take a formation slot.

Group B - slots and formation status

```
;; Formation slots
(slot-apex ?d - drone)
(slot-direction ?d - drone ?dir - direction)
(has-slot ?d - drone)
(wing-drone ?d - drone)
(formation-leader ?d - drone)
(in-formation ?d - drone)

;; Formation status
(v-formation-established)
(wings-closed)
```

`(wing-drone ?d)` is the single most important predicate in the domain. Every `forall` in the action set is written as 'either this drone is not a wing drone, or it satisfies ...' - so the domain is generic over **however many** drones carry this fact. Adding a rank means adding `wing-drone` facts in the problem file; the domain needs no change at all. It is also what `retarget_formation_problem` reads to discover the wing names.

`(slot-direction ?d ?dir)` records which compass point a wing sits towards. It is purely descriptive - no action reads it - but it is recomputed on every run, because a wing behind and to the left of a formation heading east points north-west, while the same offset heading north points north-east.

`(wings-closed)` is the interlock. `formation-cruise` **clears** it and `close-up-wings` restores it, and `formation-cruise` requires it - so the leader can never fly two legs in a row without its wings catching up.

Group C - the neighbour model [added 2026-09-16]

```
;; Neighbor communication
(wing-neighbor ?d1 - drone ?d2 - drone)
(neighbor-comm-ok ?d - drone)
```

This pair is the drone-to-drone communication model, and it is the newest part of the domain. Chapter 5 covers it in full.

`wing-neighbor` is **pairwise** and **descriptive**: it records the actual chain apex-to-rank1-to-rank2 out along each arm. No action reads it.

`neighbor-comm-ok` is **per-drone**, and it is what the actions actually check. The split is deliberate: a per-drone flag keeps each action's cost linear in the drone count, whereas checking every pair would be quadratic.

Before 2026-09-16 neither existed - each drone's readiness was judged solely on its own `communication-ok` link to the ground, with no notion of which drones actually sit next to each other in the V. The pair was added to mirror what Grid-Formation already did.

Group D - health, checks and mission status

```
;; Pre-flight check
(preflight-done ?d - drone)

;; System health
(gps-ok ?d - drone) (communication-ok ?d - drone) (drone-healthy ?d - drone)

;; Failure states
(gps-lost ?d - drone) (communication-lost ?d - drone)
(health-failure ?d - drone) (controller-notified ?d - drone)

;; Mission status
(mission-completed ?d - drone)
(formation-mission-complete)
```

Familiar from the first two plans. `(formation-mission-complete)` is nullary, describing the mission as a whole, and is set by the same action that marks every individual drone complete.

2.3 Functions

domain.pddl - lines 82-117

```
(:functions
    (battery ?d - drone) (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)

    (altitude ?d - drone) (hover-altitude ?d - drone) (target-altitude ?d - drone)

    (climb-energy ?d - drone) (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)

    (health ?d - drone) (minimum-health ?d - drone)
    (latitude ?l - location) (longitude ?l - location)

    ;; V-formation slot offsets
    (slot-along-offset ?d - drone)
    (slot-cross-offset ?d - drone)

    ;; Staggered launch timing
    (seconds-since-leader-airborne)
    (grid-launch-delay))
```

Three groups here have no Travell counterpart.

Altitude. Three fluents track a vertical profile: `hover-altitude` is where a drone sits immediately after takeoff (2 m), `target-altitude` is the cruise height (50 m), and `altitude` is the current value. `takeoff` assigns the first, `climb-to-cruise-altitude` assigns the second.

Slot offsets. `slot-along-offset` and `slot-cross-offset` record where each drone sits relative to the apex, in metres - negative along means behind, negative cross means left. These are **descriptive bookkeeping**. They are written and maintained carefully by `expand_formation_drones`, but the actual flown tracks are produced by `formation_wing_routes` from the constants in `plan_service.py`, which are kept in sync with them by hand rather than read from the file.

Launch timing. The two nullary fluents `seconds-since-leader-airborne` and `wing-launch-delay` implement the staggered launch - see actions 2 and 4.

2.4 The 14 actions

Phase 1 - getting airborne

ACTION 1 of 14 - pre-flight-check

```
(:action pre-flight-check
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d) (has-slot ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (>= (health ?d) (minimum-health ?d))
                       (> (battery ?d) (minimum-return-battery ?d)))
    :effect (preflight-done ?d))
```

The gate on takeoff. Every drone must be landed at the source, hold a slot, be fully healthy, and carry more than its return reserve. Note `(has-slot ?d)`: a drone with no assigned place in the formation cannot even start, which is what keeps an unassigned drone out of the formation entirely.

ACTION 2 of 14 - takeoff [the staggered launch]

```
(:action takeoff
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d) (preflight-done ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (or (formation-leader ?d)
                           (>= (seconds-since-leader-airborne) (wing-launch-delay))))
    :effect (and (not (landed ?d)) (airborne ?d) (flying ?d)
                 (assign (altitude ?d) (hover-altitude ?d))))
```

The disjunctive precondition is the interesting part and the reason `:disjunctive-preconditions` is required. It says: **either** you are the leader - in which case go now - **or** the leader has already been airborne for at least `wing-launch-delay` seconds.

This enforces a real launch discipline. The leader lifts off first and alone; the wings may only follow after the configured delay (5 seconds in the template). Launching a whole formation simultaneously from one point would put every aircraft in the same volume of air at the same moment.

`(assign (altitude ?d) (hover-altitude ?d))` sets the altitude outright rather than incrementing it - the drone jumps from 0 to its 2 m hover height.

ACTION 4 of 14 - hold-position [the clock]

```
(:action hold-position
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (airborne ?lead)
                       (< (seconds-since-leader-airborne) (wing-launch-delay))
                       (gps-ok ?lead) (communication-ok ?lead) (drone-healthy ?lead))
    :effect (increase (seconds-since-leader-airborne) 1))
```

This is how **time** is modelled in a domain that has no real notion of it. Each application advances the counter by one second, and the action stays applicable only while the counter is still below the delay. So the planner must apply it exactly five times to unlock the wings' takeoff.

Those five `hold-position` steps are plainly visible at steps 3 to 7 of the worked run in chapter 6. The leader is literally hovering, burning clock, while its wings wait for clearance.

Only the leader can do this - the counter is named for the **leader** being airborne, and the precondition binds `formation-leader`.

ACTION 3 of 14 - climb-to-cruise-altitude

```
(:action climb-to-cruise-altitude
    :parameters (?d - drone)
    :precondition (and (airborne ?d) (not (at-cruise-altitude ?d))
                       (< (altitude ?d) (target-altitude ?d))
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (> (battery ?d) (climb-energy ?d)))
    :effect (and (assign (altitude ?d) (target-altitude ?d))
                 (at-cruise-altitude ?d)
                 (decrease (battery ?d) (climb-energy ?d))))
```

Takes the drone from its 2 m hover height straight to the 50 m cruise altitude in one step, billing `climb-energy` (4 units) for it. The `(not (at-cruise-altitude ?d))` precondition plus the matching effect make it idempotent - it can fire at most once per drone, so a plan cannot pay the climb cost repeatedly.

Phase 2 - forming up

ACTION 5 of 14 - take-formation-slot

```
(:action take-formation-slot
    :parameters (?d - drone)
    :precondition (and (airborne ?d) (has-slot ?d) (at-cruise-altitude ?d)
                       (>= (altitude ?d) (target-altitude ?d))
                       (not (in-formation ?d))
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (in-formation ?d))
```

Each drone individually moves into its assigned place. It requires both the `at-cruise-altitude` flag **and** the numeric altitude test - belt and braces, since the flag could in principle be set while the number lagged. `(not (in-formation ?d))` makes it fire at most once per drone. Note that this action says nothing about the other drones; it is purely individual. The collective check comes next.

ACTION 6 of 14 - establish-v-formation [the collective gate]

```
(:action establish-v-formation
    :parameters (?lead - drone ?l - location)
    :precondition (and (formation-leader ?lead) (slot-apex ?lead)
                       (in-formation ?lead) (at ?lead ?l)
                       (at-cruise-altitude ?lead)
                       (>= (altitude ?lead) (target-altitude ?lead))
                       (neighbor-comm-ok ?lead)

                       (forall (?w - drone)
                           (or (not (wing-drone ?w))
                               (and (in-formation ?w)
                                    (at ?w ?l)
                                    (at-cruise-altitude ?w)
                                    (>= (altitude ?w) (target-altitude ?w))
                                    (neighbor-comm-ok ?w)))))
    :effect (and (v-formation-established) (formation-at ?l) (wings-closed)))
```

The first action that reasons about the whole swarm at once, and the template for the two that follow. It fires only when the apex **and every wing drone** are in formation, at the same location, at cruise altitude, and have healthy neighbour links.

How the `forall` works. The body reads 'either `?w` is not a wing drone, or it satisfies all of these'. That is the standard PDDL idiom for restricting a universal quantifier to a subset: drones without the `wing-drone` fact satisfy the first disjunct trivially and are effectively skipped. The leader itself is one such drone. This construction is why the domain is completely indifferent to the drone count.

`(at ?w ?l)` requires every wing to be at the **same location object** as the apex. This is the modelling abstraction that makes the plan tractable: in PDDL the entire formation occupies one location, and the real spatial offsets - the actual V shape - live entirely in the Python geometry layer.

`neighbor-comm-ok` is checked for the leader and for every wing. This is the point at which the formation refuses to form if any drone has lost contact with its neighbour.

Phase 3 - cruising as a formation

ACTION 7 of 14 - formation-cruise [the only action the Python side reads]

```
(:action formation-cruise
    :parameters (?lead - drone ?from - location ?to - location)
    :precondition (and (v-formation-established) (wings-closed)
                       (formation-leader ?lead) (slot-apex ?lead) (in-formation ?lead)
                       (formation-at ?from) (at ?lead ?from)
                       (connected ?from ?to) (safe-route ?from ?to)
                       (at-cruise-altitude ?lead)
                       (>= (altitude ?lead) (target-altitude ?lead))
                       (gps-ok ?lead) (communication-ok ?lead) (drone-healthy ?lead)
                       (neighbor-comm-ok ?lead)
                       (> (battery ?lead) (energy-required ?from ?to)))
    :effect (and (not (formation-at ?from)) (formation-at ?to)
                 (not (at ?lead ?from)) (at ?lead ?to)
                 (not (wings-closed))
                 (decrease (battery ?lead) (energy-required ?from ?to))))
```

The leader flies one leg, and the formation marker moves with it. This is the action `extract_formation_corridor` looks for; the whole flight path of the entire swarm is derived from the `?from`/`?to` arguments of these steps.

Only the leader moves. The wings stay where they were - `at ?w` is untouched. They catch up in the next action.

`(not (wings-closed))` is the interlock. Since `wings-closed` is also a precondition, the leader cannot fly a second leg until `close-up-wings` has restored it. Leg, close up, leg, close up - the formation can never string itself out.

Only the leader's battery is billed here. Each wing pays its own `cruise-leg-energy` in `close-up-wings` instead, and pays more: 14 units against the leader's 12, reflecting the extra distance a wing covers catching up and the aerodynamic penalty of not being at the front.

ACTION 8 of 14 - close-up-wings [the most intricate action in the project]

```
(:action close-up-wings
    :parameters (?lead - drone ?to - location)
    :precondition (and (v-formation-established) (not (wings-closed))
                       (formation-leader ?lead) (in-formation ?lead)
                       (formation-at ?to) (at ?lead ?to)
                       (forall (?w - drone)
                           (or (not (wing-drone ?w))
                               (and (in-formation ?w) (at-cruise-altitude ?w)
                                    (>= (altitude ?w) (target-altitude ?w))
                                    (gps-ok ?w) (communication-ok ?w) (drone-healthy ?w)
                                    (neighbor-comm-ok ?w)
                                    (> (battery ?w) (cruise-leg-energy ?w))))))
    :effect (and
        (forall (?w - drone) (forall (?x - location)
            (when (and (wing-drone ?w) (at ?w ?x))
                  (not (at ?w ?x)))))
        (forall (?w - drone)
            (when (wing-drone ?w) (at ?w ?to)))
        (forall (?w - drone)
            (when (wing-drone ?w)
                  (decrease (battery ?w) (cruise-leg-energy ?w))))
        (wings-closed)))
```

Moves **every** wing drone to the leader's new location in a single action, and bills each of them for the leg. It is worth reading its effect carefully, because it demonstrates the one genuinely advanced PDDL construction in the codebase.

The first effect is a doubly-nested conditional delete. For every wing drone `?w` and every location `?x`, if `?w` is currently at `?x`, delete that fact. The nested `forall` over **locations** is necessary because PDDL has no way to say 'delete wherever it happens to be' - the effect must range over every possible location and delete conditionally.

The second effect adds them all at the new location, and the third bills each one's battery. Three separate `forall` blocks rather than one, because each does a different kind of thing - delete, add, and decrease.

The precondition checks every wing's battery before any of them move. This is an all-or-nothing manoeuvre: the formation does not partially close up. If one wing is too low on battery, the whole action is inapplicable and the formation is stuck - which is a correct model of a rigid formation, since the group can only go as far as its weakest member.

And it checks every wing's `neighbor-comm-ok`. A wing that has lost its neighbour link blocks the entire formation from advancing.

ACTION 9 of 14 - complete-formation-mission

```
(:action complete-formation-mission
    :parameters (?lead - drone ?dest - location)
    :precondition (and (destination ?dest) (v-formation-established)
                       (formation-at ?dest)
                       (formation-leader ?lead) (slot-apex ?lead)
                       (in-formation ?lead) (at ?lead ?dest)
                       (forall (?w - drone)
                           (or (not (wing-drone ?w))
                               (and (in-formation ?w) (at ?w ?dest)))))
    :effect (and (mission-completed ?lead)
                 (forall (?w - drone)
                     (when (wing-drone ?w) (mission-completed ?w)))
                 (formation-mission-complete)))
```

Closes the mission once the entire formation - apex and every wing - is at the destination and still in formation. One action marks the leader complete explicitly and every wing complete through a conditional effect, then sets the nullary mission flag. This is what satisfies the goal's per-drone `mission-completed` conjuncts in a single step, however many drones there are.

Phase 4 - fault detection

Four detectors, all sharing the same shape: require `(flying ?d)`, set a failure fact, clear `flying`, and - new in this domain - clear `in-formation`, which ejects the faulty drone from the formation and thereby blocks every collective action above.

ACTIONS 10, 11 and 13 of 14 - the three inherited detectors

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d)) (not (in-formation ?d))))

(:action detect-communication-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (communication-ok ?d)))
    :effect (and (communication-lost ?d) (not (flying ?d)) (not (in-formation ?d))))

(:action detect-health-failure
    :parameters (?d - drone)
    :precondition (and (flying ?d) (< (health ?d) (minimum-health ?d)))
    :effect (and (health-failure ?d) (not (flying ?d)) (not (in-formation ?d))))
```

The `(not (in-formation ?d))` effect is the important addition over Travell. Because every collective action requires `in-formation` for every wing drone, one faulting drone immediately prevents the formation from establishing, closing up, or completing. The formation is modelled as genuinely rigid: it does not fly on without a member.

ACTION 12 of 14 - detect-neighbor-link-loss [added 2026-09-16]

```
(:action detect-neighbor-link-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d)
                       (not (neighbor-comm-ok ?d)))
    :effect (and (communication-lost ?d)
                 (not (flying ?d))
                 (not (in-formation ?d))))
```

The fourth detector, and the only one with no Travell counterpart. It fires when an airborne drone has lost contact with its **neighbours in the V** - as opposed to `detect-communication-loss`, which is about the link to the **ground controller**.

The two failures are modelled as equally serious: both set the same `communication-lost` fact and both eject the drone from the formation. In a tight formation that is the right call - a drone that cannot hear the aircraft 20 m off its wing is a hazard whether or not the ground station can still hear it.

This action, together with the `neighbor-comm-ok` checks in actions 6, 7 and 8, was added on 2026-09-16 to mirror what Grid-Formation already had. It was verified against ENHSP directly for 3, 5 and 7-drone formations, including through the full retarget-and-expand pipeline.

ACTION 14 of 14 - notify-ground-controller

```
(:action notify-ground-controller
    :parameters (?d - drone ?c - controller)
    :precondition (and (or (gps-lost ?d) (communication-lost ?d) (health-failure ?d)))
    :effect (controller-notified ?d))
```

The single upward message, as in Travell. All four detectors funnel into it - note that both communication detectors set `communication-lost`, so a neighbour-link failure is reported through the same channel. Unlike Travell there are no downward controller orders in this domain: a fault is reported and the formation is broken, but nothing responds.

3. problem.pddl explained

problem.pddl - the objects

```
(define (problem v-formation-mission-001)
    (:domain v-formation-drone-mission)

    (:objects
        drone-lead  - drone
        drone-left  - drone
        drone-right - drone
        ground-controller - controller
        source      - location
        destination - location

        north  north-east  east  south-east
        south  south-west  west  north-west  - direction
    ))
```

Three drones: the apex and one wing on each side - the minimum V, and the template from which `expand_formation_drones` grows every larger one. It clones `drone-left` and `drone-right` to produce `drone-left2`, `drone-right2`, and so on.

The eight compass directions are declared as objects so that `slot-direction` has something to point at. They correspond exactly to the `_COMPASS_POINTS` tuple in `engine/pddl_problem.py`, listed there in bearing order from north in 45-degree steps.

Only two locations. Unlike Travell there are no template waypoints at all - the V flies source to destination directly unless a marked restriction forces via-points to be generated.

problem.pddl - starting state and slot assignment

```
;; All drones start landed at the source
(at drone-lead source) (at drone-left source) (at drone-right source)
(landed drone-lead) (landed drone-left) (landed drone-right)
(= (altitude drone-lead) 0) (= (altitude drone-left) 0) (= (altitude drone-right) 0)

;; Direct route between source and destination
(connected source destination)  (connected destination source)
(safe-route source destination) (safe-route destination source)

;; Assign the leader and two wing positions
(formation-leader drone-lead)
(slot-apex drone-lead)
(slot-direction drone-left  north-west)
(slot-direction drone-right south-west)
(wing-drone drone-left)
(wing-drone drone-right)
(has-slot drone-lead) (has-slot drone-left) (has-slot drone-right)
```

All three drones begin landed at the source at zero altitude. The direct route is declared legal in both directions - and unlike Travell's template, where the direct leg was deliberately **not** a safe-route, here it is. The V has no built-in detour shape; a detour is only ever generated at run time.

`drone-lead` carries both `formation-leader` and `slot-apex`, and notably does **not** carry `wing-drone` - which is what excludes it from every `forall` over wings.

The two `slot-direction` values, north-west and south-west, describe the template's own westward-heading scenario. `retarget_formation_problem` recomputes both from the newly picked bearing on every run.

problem.pddl - the slot offsets

```
;; V-formation slot offsets
(= (slot-along-offset drone-lead)    0)
(= (slot-cross-offset drone-lead)    0)
(= (slot-along-offset drone-left)   -17.320508)
(= (slot-cross-offset drone-left)   -10)
(= (slot-along-offset drone-right)  -17.320508)
(= (slot-cross-offset drone-right)   10)
```

The V's geometry, in metres. The apex sits at the origin of the formation frame. Both wings sit 17.320508 m **behind** it - negative along means behind - and 10 m to their own side, left negative and right positive.

As section 1.1 showed, those two numbers give a hypotenuse of exactly 20 m, putting the leader and both rank-1 wings at the corners of an equilateral triangle.

These facts **describe** the geometry but do not **drive** it. The flown tracks come from `FORMATION_BACK_M` and `FORMATION_SIDE_M` in `services/plan_service.py`, which hold the same two numbers and are kept in sync with these by hand - `formation_wing_routes` works from the apex's actual flown path, not from the problem file's fluents.

A note on formatting that matters: each of these is written on **one line**. The identical facts in the Grid-Formation template were once split across three lines each, which broke the line-based cloner and made every grid larger than 1x2 unsolvable. This file's one-line convention is the one the grid template was corrected to follow.

problem.pddl - the neighbour chain and the launch clock

```
;; Neighbor communication - each drone talks to its own left/right
;; neighbor along its wing (here, rank 1 either side of the leader)
(wing-neighbor drone-lead drone-left)
(wing-neighbor drone-left drone-lead)
(wing-neighbor drone-lead drone-right)
(wing-neighbor drone-right drone-lead)

(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-left)
(neighbor-comm-ok drone-right)

;; Leader waits 5 seconds before wing drones take off
(= (seconds-since-leader-airborne) 0)
(= (wing-launch-delay) 5)
```

For the 3-drone template the chain is trivial: the leader is the neighbour of both wings. Each link is declared symmetrically, in both directions.

For anything larger, `expand_formation_drones` **strips these facts entirely** and rebuilds the full chain - see section 4.3 for why cloning them would produce a wrong topology.

The launch clock starts at zero against a delay of 5, which is what forces exactly five `hold-position` actions into every plan.

problem.pddl - health, battery and the flight profile

```
;; All drones start with normal health and system status
(gps-ok drone-lead) (communication-ok drone-lead) (drone-healthy drone-lead)
...
(= (health drone-lead) 100)  (= (minimum-health drone-lead) 40)
(= (battery drone-lead) 100) (= (max-battery drone-lead) 100)
(= (minimum-return-battery drone-lead) 30)

;; Drones start at 2 m and climb to 50 m
(= (hover-altitude drone-lead)  2)   (= (target-altitude drone-lead)  50)
(= (climb-energy drone-lead)  4)

;; Battery used by each drone for one cruise leg
(= (cruise-leg-energy drone-lead)  12)
(= (cruise-leg-energy drone-left)  14)
(= (cruise-leg-energy drone-right) 14)
```

Standard healthy starting values. The altitude profile - 2 m hover, 50 m cruise, 4 units to climb - is identical for all three drones.

The one asymmetry is `cruise-leg-energy`: 12 for the leader, 14 for each wing. The wings genuinely work harder, both because closing up covers more ground than the leader's own leg and because only the lead aircraft gets the aerodynamic benefit of being at the front. This is the only place in any of the four templates where drones are given different performance figures.

problem.pddl - the goal

```
(:goal (and
    (v-formation-established)
    (in-formation drone-lead) (in-formation drone-left) (in-formation drone-right)

    (at-cruise-altitude drone-lead) (at-cruise-altitude drone-left)
    (at-cruise-altitude drone-right)
    (>= (altitude drone-lead) 50) (>= (altitude drone-left) 50)
    (>= (altitude drone-right) 50)

    (formation-at destination)
    (at drone-lead destination) (at drone-left destination)
    (at drone-right destination)

    (formation-mission-complete)
    (mission-completed drone-lead) (mission-completed drone-left)
    (mission-completed drone-right)))
```

Much richer than Travell's two conjuncts. It demands not merely arrival but that the formation was **established**, that every drone is **still in formation** on arrival, and that every drone is at cruise altitude both by flag and by number.

Requiring `in-formation` for every drone **in the goal** is what makes a fault genuinely fatal to the mission: any detector that fires clears that flag, and no action can restore it once the drone has stopped flying.

`expand_formation_drones` clones the per-drone conjuncts for every new rank, so a 7-drone V's goal carries the same five conjuncts for each of its seven drones.

4. Every Python function in the V-Formation path

4.1 The constants and the naming

FORMATION_BACK_M, FORMATION_SIDE_M and _formation_drone_names(ranks)

```
FORMATION_BACK_M = 17.320508
FORMATION_SIDE_M = 10.0

def _formation_drone_names(ranks: int) -> list[str]:
    names = ["drone-lead"]
    for r in range(1, ranks + 1):
        suffix = "" if r == 1 else str(r)
        names.append(f"drone-left{suffix}")
        names.append(f"drone-right{suffix}")
    return names
```

The two constants hold the same numbers as the problem file's slot offsets, and the comment above them is explicit that they must be kept in sync by hand - `formation_wing_routes` works from the apex's actual flown path, not from the PDDL fluents, so the two representations are independent.

`_formation_drone_names` produces the names in the order the GUI's checked drones are assigned to slots: apex first, then each rank's left and right **interleaved**.

That interleaving is a deliberate usability choice, spelled out in the docstring: listing every left rank and then every right rank would mean the second checked drone ends up at the tip of the left wing on a large formation. Interleaving by rank means the first checked drones always land nearest the apex, whatever the size of the V.

Note the empty suffix for rank 1, which keeps the template's own names `drone-left` and `drone-right` rather than renaming them to `drone-left1`.

4.2 Compass bearings and slot directions

_COMPASS_POINTS, _bearing_deg and _nearest_compass_point

```
_COMPASS_POINTS = ("north", "north-east", "east", "south-east",
                   "south", "south-west", "west", "north-west")

def _bearing_deg(a: LatLon, b: LatLon) -> float:
    east, north = _to_local(b, a)
    return math.degrees(math.atan2(east, north)) % 360.0

def _nearest_compass_point(bearing_deg: float) -> str:
    return _COMPASS_POINTS[round(bearing_deg / 45.0) % 8]
```

`_bearing_deg` gives the compass bearing from one point to another - 0 is north, increasing clockwise. Note the argument order inside `atan2`: `(east, north)` rather than the usual `(y, x)`. That swap is exactly what converts a mathematical angle, measured counter-clockwise from east, into a compass bearing measured clockwise from north.

`_nearest_compass_point` snaps a bearing to one of eight labels by dividing by 45 and rounding. The modulo 8 handles the wrap: a bearing of 350 degrees rounds to index 8, which wraps back to north.

_wing_slot_direction and _set_slot_direction

```
def _wing_slot_direction(travel_bearing_deg, along_offset_m, cross_offset_m) -> str:
    rad = math.radians(travel_bearing_deg)
    east  = along_offset_m * math.sin(rad) + cross_offset_m * math.cos(rad)
    north = along_offset_m * math.cos(rad) - cross_offset_m * math.sin(rad)
    return _nearest_compass_point(math.degrees(math.atan2(east, north)) % 360.0)

def _set_slot_direction(problem_text, drone_name, direction_name) -> str:
    def _repl(match):
        if match.group(2) != drone_name: return match.group(0)
        return f"{match.group(1)}{direction_name}{match.group(4)}"
    return _SLOT_DIRECTION_FACT.sub(_repl, problem_text)
```

`_wing_slot_direction` answers: given a formation travelling on a particular bearing, which real compass point does a wing's fixed offset actually point towards?

It rotates the formation-frame offset into the world frame. The offset is expressed **relative to the direction of travel** - along is backwards, cross is sideways - so it must be rotated by the travel bearing to become a real-world direction. The example in the `retarget_formation_problem` docstring makes it concrete: a wing behind and to the left of a formation heading east points north-west, but the same offset on a formation heading north points north-east instead.

`_set_slot_direction` rewrites just the direction argument of a `slot-direction` fact for one named drone, leaving every other drone's fact untouched by returning the unmodified match.

_unary_fact_value and _set_unary_fact_value

```
def _unary_fact_value(problem_text, predicate, obj) -> float | None:
    pattern = re.compile(
        rf"\(=\s*\({re.escape(predicate)}\s+{re.escape(obj)}\)\s*([-\d.eE]+)\s*\)")
    match = pattern.search(problem_text)
    return float(match.group(1)) if match else None

def _set_unary_fact_value(problem_text, predicate, obj, value) -> str:
    pattern = re.compile(
        rf"(\(=\s*\({re.escape(predicate)}\s+{re.escape(obj)}\)\s*)[-\d.eE]+(\s*\))")
    return pattern.sub(lambda m: f"{m.group(1)}{value:.6f}{m.group(2)}",
                       problem_text, count=1)
```

The one-argument counterparts to `_fact_value` and `_set_fact_value`, for per-drone fluents such as `slot-along-offset`. They are used to read the template's rank-1 offsets and to write each new rank's scaled values. Six decimal places preserves the 17.320508 figure exactly.

4.3 Growing the V

_strip_pair_facts(problem_text, predicate)

```
def _strip_pair_facts(problem_text: str, predicate: str) -> str:
    return re.sub(rf"[ \t]*\({re.escape(predicate)}\s+\S+\s+\S+\)\n?", "", problem_text)
```

Deletes every pairwise fact of a given predicate. It is shared by both formation expanders - `wing-neighbor` here, `grid-neighbor` in the grid - to clear out the template's own hardcoded rank-1 pairs before the complete set is computed fresh. The next function explains why that is essential rather than merely tidy.

expand_formation_drones - steps 1: validation and new objects

```
def expand_formation_drones(problem_text, drone_count, *,
                            left_wing_name="drone-left",
                            right_wing_name="drone-right") -> str:
    if drone_count < 3 or drone_count % 2 == 0:
        raise ValueError(f"v-formation drone_count must be odd and >= 3, got {drone_count}")
    ranks = (drone_count - 1) // 2
    if ranks <= 1: return problem_text

    base_along = {name: _unary_fact_value(problem_text, "slot-along-offset", name)
                  for name in (left_wing_name, right_wing_name)}
    base_cross = {name: _unary_fact_value(problem_text, "slot-cross-offset", name)
                  for name in (left_wing_name, right_wing_name)}
    if any(v is None for v in (*base_along.values(), *base_cross.values())):
        raise ValueError("template has no slot-along-offset/slot-cross-offset for ...")

    # 1. Objects: one "<name><rank> - drone" line per new rank per wing
    obj_open, obj_close = _section_span(problem_text, "objects")
    extra_objects = "".join(f"\n        {name}{r} - drone"
                            for r in range(2, ranks + 1)
                            for name in (left_wing_name, right_wing_name))
    result = problem_text[:obj_close] + extra_objects + "\n    " + problem_text[obj_close:]

    left_ranks  = [f"{left_wing_name}{r}"  for r in range(2, ranks + 1)]
    right_ranks = [f"{right_wing_name}{r}" for r in range(2, ranks + 1)]
```

The validation is strict and deliberate. A V must have an odd count of at least three: one apex plus a matched pair per rank. An even count would leave one wing longer than the other. The `ValueError` is caught by `_Worker.run` and reported as a failed run rather than crashing the application.

The template's own rank-1 offsets are read first, because every further rank's offsets are computed by scaling them.

`ranks <= 1` returns the template untouched - a 3-drone V is exactly what the template already describes.

expand_formation_drones - steps 2 and 3: clone, then rescale

```
    # 2. Clone every :init fact mentioning a rank-1 wing, once per new rank
    result = _clone_named_lines(result, _section_span(result, "init"),
                                left_wing_name, left_ranks)
    result = _clone_named_lines(result, _section_span(result, "init"),
                                right_wing_name, right_ranks)
    for r, name in zip(range(2, ranks + 1), left_ranks):
        result = _set_unary_fact_value(result, "slot-along-offset", name,
                                       base_along[left_wing_name] * r)
        result = _set_unary_fact_value(result, "slot-cross-offset", name,
                                       base_cross[left_wing_name] * r)
    for r, name in zip(range(2, ranks + 1), right_ranks):
        result = _set_unary_fact_value(result, "slot-along-offset", name,
                                       base_along[right_wing_name] * r)
        result = _set_unary_fact_value(result, "slot-cross-offset", name,
                                       base_cross[right_wing_name] * r)

    # 3. Clone the goal's per-drone conjuncts, inside its (and ...)
    result = _clone_named_lines(result, _goal_and_span(result), left_wing_name, left_ranks)
    result = _clone_named_lines(result, _goal_and_span(result), right_wing_name, right_ranks)
```

Every initial-state fact mentioning a rank-1 wing is cloned for each new rank - health, battery, altitudes, `wing-drone`, `has-slot`, `slot-direction` - and then the **only** two facts that differ between ranks are corrected: the along and cross offsets, each multiplied by the rank number.

Why `slot-direction` needs no correction. The docstring makes the point neatly: the compass direction depends only on the *ratio* of along to cross offset, and scaling both by the same rank number leaves that ratio unchanged. So cloning rank 1's direction fact onto every further rank is already correct - every drone on one arm points the same way.

Why scaling by rank produces a straight arm. Rank 2 sits twice as far back and twice as far out as rank 1, rank 3 three times - so every consecutive pair along an arm is the same 20 m apart, and the arm reads as a straight line running out from the apex.

Step 3 clones the goal conjuncts **inside** the goal's `(and ...)`, using the same `_goal_and_span` helper trick as the point-to-point expander, so the result stays a single well-formed goal expression.

expand_formation_drones - step 4: rebuilding the neighbour chain

```
    # 4. Neighbor links: leader <-> rank 1, then rank r <-> rank r+1 out
    #    to the tip, one symmetric pair per link.
    result = _strip_pair_facts(result, "wing-neighbor")
    chain_facts = []
    for chain in (["drone-lead", left_wing_name,  *left_ranks],
                  ["drone-lead", right_wing_name, *right_ranks]):
        for a, b in zip(chain, chain[1:]):
            chain_facts.append(f"\n        (wing-neighbor {a} {b})")
            chain_facts.append(f"\n        (wing-neighbor {b} {a})")
    result = _add_init_facts(result, "".join(chain_facts))
    return result
```

The step that most repays understanding, because it exists to **undo** something step 2 got wrong.

The bug it prevents. Step 2 cloned every line mentioning `drone-left`, which includes the template's `(wing-neighbor drone-lead drone-left)`. Cloned for rank 3, that becomes `(wing-neighbor drone-lead drone-left3)` - wiring the **tip** of the wing directly to the leader. Every rank would end up a neighbour of the leader and of nobody else, which is not a chain at all and misdescribes the formation completely.

The fix. Strip every `wing-neighbor` fact and lay down the correct chain fresh: leader to rank 1, rank 1 to rank 2, rank 2 to rank 3, and so on out to the tip, separately for each arm. Each link is written symmetrically in both directions.

The result for a 7-drone V is two chains of three links each: lead-left-left2-left3 and lead-right-right2-right3. Each drone has at most two neighbours - one inboard, one outboard - and the tips have only one.

4.4 Retargeting the V

retarget_formation_problem(...) [the main entry point]

```
_WING_DRONE_FACT = re.compile(r"\(wing-drone\s+(\S+)\)")

def retarget_formation_problem(problem_text, new_source, new_destination, *,
                               no_fly_zone=None, no_fly_buffer_m=NO_FLY_BUFFER_M,
                               source_name="source", destination_name="destination",
                               left_wing_name="drone-left",
                               right_wing_name="drone-right") -> str:
    locations = read_locations(problem_text)
    if source_name not in locations or destination_name not in locations:
        raise ValueError(...)
    transform = _similarity_transform(locations[source_name], locations[destination_name],
                                      new_source, new_destination)
    moved = {name: transform(point) for name, point in locations.items()}
    moved[source_name] = new_source
    moved[destination_name] = new_destination
    result = problem_text
    for name, point in moved.items():
        result = _set_latlon(result, name, point)

    for a, b in _read_edges(problem_text):        # recompute distance/energy
        ...                                        # exactly as retarget_problem does

    if no_fly_zone and len(no_fly_zone) >= 3:
        seg_a = _to_local(new_source, new_source)
        seg_b = _to_local(new_destination, new_source)
        polygon_local = _convex_hull([_to_local(p, new_source) for p in no_fly_zone])
        endpoint_inside = (_point_in_polygon(seg_a, polygon_local)
                           or _point_in_polygon(seg_b, polygon_local))
        if len(polygon_local) >= 3 and not endpoint_inside:
            routing_polygon = _offset_polygon(polygon_local, no_fly_buffer_m)
            if (_point_in_polygon(seg_a, routing_polygon)
                    or _point_in_polygon(seg_b, routing_polygon)):
                routing_polygon = polygon_local
            if _segment_crosses_polygon(seg_a, seg_b, routing_polygon):
                detour_local = _shortest_detour(seg_a, seg_b, routing_polygon)
                via_points = [_from_local(p, new_source) for p in detour_local[1:-1]]
                if via_points:
                    result = _set_detour_route(result, source_name, destination_name,
                                               new_source, new_destination, via_points)

    bearing = _bearing_deg(new_source, new_destination)
    wing_names = _WING_DRONE_FACT.findall(problem_text) or [left_wing_name, right_wing_name]
    for wing_name in wing_names:
        along = _unary_fact_value(problem_text, "slot-along-offset", wing_name)
        cross = _unary_fact_value(problem_text, "slot-cross-offset", wing_name)
        if along is not None and cross is not None:
            result = _set_slot_direction(result, wing_name,
                                         _wing_slot_direction(bearing, along, cross))
    return result
```

Structurally very close to `retarget_problem`, with two meaningful differences.

Difference 1 - there is no `_set_direct_route_only` branch. A Travell template carries a built-in detour that must be pruned when nothing blocks the route; this template's only declared route *is* the direct one, so when nothing blocks it there is simply nothing to do. Only the detour branch exists.

Difference 2 - the slot directions are recomputed at the end. Every wing's compass label is recalculated from the **new** source-to-destination bearing. The offsets themselves never change - the V is the same shape wherever it flies - but which compass point that shape's wings happen to face does.

Discovering the wings. `_WING_DRONE_FACT.findall` reads the wing names straight out of the problem's own `(wing-drone ...)` facts, so however many ranks `expand_formation_drones` created, all of them are updated. The fallback to the two default names covers a template predating the predicate.

The buffer is a parameter here, not a constant. That matters: the caller pads it, because only the apex's centreline is planned against the zone while the wings ride out to the side of it.

4.5 Deriving the wing tracks

_unit(v)

```
def _unit(v: tuple[float, float]) -> tuple[float, float] | None:
    norm = math.hypot(*v)
    return (v[0] / norm, v[1] / norm) if norm > 1e-9 else None
```

Normalises a vector to unit length, returning `None` for a vector too short to have a meaningful direction. That `None` is used as a signal by the caller rather than being an error - it means 'these two waypoints coincide, so there is no heading here'.

formation_member_route(lead_route, along_m, cross_m) [the geometry that makes the V]

```
def formation_member_route(lead_route, *, along_m, cross_m) -> list[LatLon]:
    if len(lead_route) < 2: return list(lead_route)
    n = len(lead_route)
    route = []
    for i, here in enumerate(lead_route):
        incoming = _unit(_to_local(here, lead_route[i-1])) if i > 0 else None
        outgoing = _unit(_to_local(lead_route[i+1], here)) if i + 1 < n else None

        fwd = None
        if incoming and outgoing:
            fwd = _unit((incoming[0] + outgoing[0], incoming[1] + outgoing[1]))
        if fwd is None:
            fwd = incoming or outgoing
        if fwd is None:
            route.append(here); continue

        port = (-fwd[1], fwd[0])                      # 90 deg left of forward
        back = (-fwd[0] * along_m, -fwd[1] * along_m)
        route.append(_from_local(
            (back[0] + port[0] * cross_m, back[1] + port[1] * cross_m), here))
    return route
```

Produces one formation member's track, held parallel to the leader's. For each of the leader's waypoints it emits a corresponding point offset `along_m` behind and `cross_m` to the right - so the member-to-leader spacing is the fixed formation distance **everywhere** along the route, at the source and destination alike, not only once airborne.

The turn bisector is the subtle part, and the docstring dwells on it for good reason. At an interior waypoint - a detour bend - 'forward' is taken as the sum of the incoming and outgoing unit directions, which bisects the turn.

The tempting shortcut is to use the straight line from the point **before** the bend to the point **after** it. That is wrong, and wrong in a way that is easy to miss: for a single bend (source, via, destination) that line collapses straight back to the original source-to-destination bearing no matter how far the via-point swings out. A member would then hold its pre-detour heading exactly at the bend - pinching in towards the lead track or crossing it - rather than turning with the formation.

At an endpoint there is only one leg, so that leg's own direction is used.

`port` is the unit vector 90 degrees to the left of forward, obtained by the standard rotation `(x, y) -> (-y, x)`. The final point is the leader's position, pushed backwards along `fwd` and sideways along `port`.

The returned route always has the same number of points as the leader's, which is what lets every drone's track be flown as a parallel sequence.

formation_wing_routes(lead_route, back_m, side_m)

```
def formation_wing_routes(lead_route, *, back_m, side_m
                          ) -> tuple[list[LatLon], list[LatLon]]:
    left  = formation_member_route(lead_route, along_m=back_m, cross_m=side_m)
    right = formation_member_route(lead_route, along_m=back_m, cross_m=-side_m)
    return left, right
```

A V's two wings are always a symmetric pair around the lead track, so this is simply two calls to `formation_member_route` with opposite cross offsets. The grid formation cannot use this shortcut - its members do not come in matched pairs - which is precisely why the single-offset `formation_member_route` was factored out of it.

extract_formation_corridor(steps) [engine/pddl_planner.py]

```
def extract_formation_corridor(steps: list[PlanStep]) -> list[str]:
    legs = [step.args[1:3]
            for step in sorted(steps, key=lambda s: s.index)
            if step.action == "formation-cruise" and len(step.args) >= 3]
    if not legs: return []
    corridor = [legs[0][0]]
    corridor.extend(to for _from, to in legs)
    return corridor
```

Pulls the apex's route out of the plan. Unlike `extract_route` it takes no drone argument, because there is only ever **one** corridor in a formation plan - only the leader moves in a `formation-cruise` action. The chaining is the same as the other two extractors: the first leg's origin followed by every leg's destination. It is shared unchanged with Grid-Formation, whose domain uses the same action name.

4.6 Orchestration

_Worker._run_formation(...) - the V-Formation run, start to finish

```
def _run_formation(self, plan_name, source, destination,
                   no_fly_zone=None, drone_count=3) -> PlanRunResult:
    ranks = max(1, (drone_count - 1) // 2)
    domain_path, template_path = self._plan_files(plan_name)
    template_text = template_path.read_text(encoding="utf-8")
    template_text = expand_formation_drones(template_text, max(3, drone_count))
    concrete_text = retarget_formation_problem(
        template_text,
        PddlLatLon(lat=source.lat, lon=source.lon),
        PddlLatLon(lat=destination.lat, lon=destination.lon),
        no_fly_zone=(...),
        no_fly_buffer_m=NO_FLY_BUFFER_M + FORMATION_SIDE_M * ranks)

    run_dir = self._start_run(plan_name, concrete_text)
    steps, stdout = run_enhsp(domain_path, run_dir / "problem.pddl")
    (run_dir / "plan.txt").write_text(stdout, encoding="utf-8")

    corridor = extract_formation_corridor(steps)
    if not corridor:
        raise PlanNotFound("ENHSP found a plan but it contains no 'formation-cruise' legs.")
    locations = read_locations(concrete_text)
    lead_route = [PddlLatLon(lat=locations[n].lat, lon=locations[n].lon) for n in corridor]

    routes = [lead_route]
    for r in range(1, ranks + 1):
        left_route, right_route = formation_wing_routes(
            lead_route, back_m=FORMATION_BACK_M * r, side_m=FORMATION_SIDE_M * r)
        routes.append(left_route)
        routes.append(right_route)
    per_drone_waypoints = dict(zip(_formation_drone_names(ranks),
                                   (_as_gui(r) for r in routes)))

    return PlanRunResult(plan_name=plan_name, steps=steps,
                         waypoints=_as_gui(lead_route), location_names=corridor,
                         run_dir=run_dir, per_drone_waypoints=per_drone_waypoints)
```

Expand before retarget, like Search and unlike Travell - the retargeting step needs to know which drones are wings so that it can recompute all their slot directions.

The padded no-fly buffer is the key line. ENHSP plans only the apex's centreline, but the outermost wings ride `FORMATION_SIDE_M * ranks` out to either side of it. Buffering by the base 10 m alone would clear the apex while flying the wing tips through the marked area. Padding the buffer by the formation's own lateral reach ensures the **whole V** clears the zone, not just its centreline.

The wing routes are built rank by rank. `formation_wing_routes` is called once per rank with `back_m` and `side_m` both scaled by the rank number, producing a left and a right track each time. The routes list is assembled in exactly the order `_formation_drone_names` produces names, so the `zip` pairs them correctly.

The result carries both forms. `waypoints` holds the leader's route for any display code that expects a single path, while `per_drone_waypoints` holds all of them - the same shape the Search plan produces, so the swarm flies concurrently through the same per-drone execution path.

The `max(3, drone_count)` floor means a request for fewer than three drones is quietly raised to the minimum V rather than rejected.

5. How the drones communicate

This is the first of the two plans with a genuine drone-to-drone model, so it is worth separating the two channels carefully - they are different links with different failure modes.

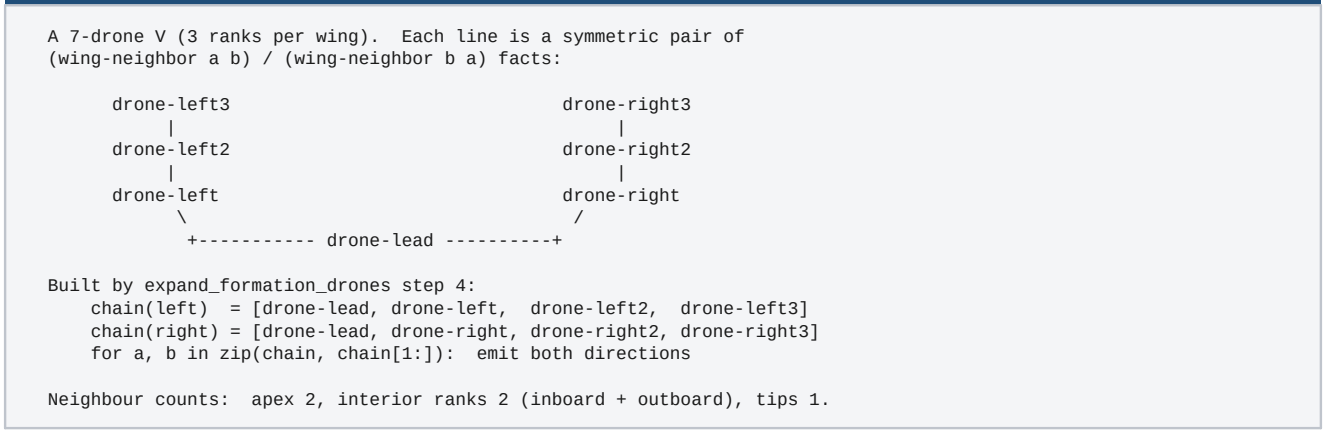
5.1 Two distinct channels

Fact	What it means	Checked by
`(communication-ok ?d)`	This drone's radio link to the GROUND CONTROLLER is healthy.	pre-flight-check, takeoff, climb-to-cruise-altitude, hold-position, take-formation-slot, formation-cruise, close-up-wings
`(neighbor-comm-ok ?d)`	This drone's links to its NEIGHBOURS IN THE V are healthy.	establish-v-formation, formation-cruise, close-up-wings
`(wing-neighbor ?d1 ?d2)`	?d1 and ?d2 are adjacent along a wing. Descriptive only - no action reads it.	nothing

The division of labour is clear from the third column. The ground link is needed for **every** individual action, including getting off the ground. The neighbour link is needed only for the three **collective** actions - forming up, cruising, and closing up - because those are the points at which drones must actually coordinate with each other.

5.2 The neighbour topology

The wing-neighbor chain



Each drone talks only to the aircraft immediately inboard and outboard of it along its own arm. There is no link across the V - `drone-left` and `drone-right` are not neighbours - and no link to distant ranks. This models what real formation flying requires: each pilot keeps station on the aircraft next to them, not on all the others.

The topology is purely descriptive. What the actions actually check is the per-drone `neighbor-comm-ok` flag, and the reason is efficiency: a per-drone check keeps each action's cost linear in the drone count, whereas verifying every declared pair would be quadratic and would multiply the number of grounded actions ENHSP must consider.

5.3 What happens when a link fails

Failure propagation

```
GROUND LINK LOST                      NEIGHBOUR LINK LOST
(communication-ok false)              (neighbor-comm-ok false)
        |                                      |
        v                                      v
detect-communication-loss            detect-neighbor-link-loss
        |                                      |
        +--------------+-----------------------+
                       |
                       v
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))       <-- ejected from the formation
                       |
      +----------------+----------------------------+
      |                                             |
      v                                             v
notify-ground-controller              every collective action is now BLOCKED:
(controller-notified ?d)                establish-v-formation  - forall fails
                                        close-up-wings         - forall fails
                                        complete-formation-    - forall fails
                                          mission
                                      and the GOAL itself requires
                                        (in-formation ?d) for every drone
```

Both failures are treated identically and both are **fatal to the mission**. Because every collective action's `forall` requires `in-formation` for every wing drone, and because the goal itself demands it for every drone, a single lost link anywhere in the V makes the mission unachievable. No action can restore `in-formation` once the drone has stopped flying.

That is the correct model for a rigid formation: the V does not fly on with a gap in it. It does, however, mean the domain has no **degraded** mode - there is no action to reconfigure the formation around a lost member, and no downward controller order to respond with. The model detects and reports; it does not recover.

5.4 At runtime: still a star through the GCS

The runtime transport

```
                   GROUND CONTROL STATION
                     (engine/gcs.py)
                  one UDP socket, port 14690
                             |
      +----------------------+----------------------+
      |                      |                      |
   udp:14701              udp:14702              udp:1470N
   drone-lead             drone-left             drone-right ...

  GCS  ------ COMMAND (JSON/UDP) ------>  drone
  GCS  <-- TELEMETRY / HEARTBEAT / ACK --  drone
  drone  <--------- X ---------->  drone      (no such link exists)
```

This is the point at which the model and the implementation diverge, and it should be stated plainly in any write-up. The `wing-neighbor` chain and `neighbor-comm-ok` flag describe a **peer-to-peer** communication structure. The runtime has no peer-to-peer transport at all: grepping the codebase for peer links finds `neighbor` only in the PDDL-generating modules, never in the networking ones.

Every drone binds its own UDP port at `14700 + sysid` and speaks only to the GCS, which holds one socket for the entire swarm, keeps the latest telemetry and heartbeat per drone, and fans commands back out. Formation cohesion at runtime is achieved not by drones talking to each other but by each flying its own precomputed parallel track - the tracks `formation_member_route` generated, which are parallel by construction and therefore keep station without any negotiation.

So the accurate summary is: **the neighbour model is a planning-time abstraction for what a real formation-flying implementation would need.** It lets the planner refuse to launch a formation whose members could not maintain contact, and it documents the topology such a system would use. Implementing it would mean adding a peer transport alongside the existing star.

6. A worked run

A real 7-drone V-Formation run from `plans/vformation/runs/`, complete:

ENHSP output - 7 drones, 3 ranks per wing, direct route

```
Problem Solved
Found Plan:
 0.0: (pre-flight-check drone-lead source)
 1.0: (takeoff drone-lead source)
 2.0: (climb-to-cruise-altitude drone-lead)
 3.0: (hold-position drone-lead)          <-- the leader burns the launch clock
 4.0: (hold-position drone-lead)
 5.0: (hold-position drone-lead)
 6.0: (hold-position drone-lead)
 7.0: (hold-position drone-lead)          <-- 5 x 1 s = wing-launch-delay
 8.0: (pre-flight-check drone-left source)
 9.0: (takeoff drone-left source)
10.0: (climb-to-cruise-altitude drone-left)
11.0: (pre-flight-check drone-right2 source)
12.0: (takeoff drone-right2 source)
13.0: (climb-to-cruise-altitude drone-right2)
...                                    (the other wings, same 3 steps each)
26.0: (take-formation-slot drone-lead)
27.0: (take-formation-slot drone-left2)
28.0: (take-formation-slot drone-left)
29.0: (take-formation-slot drone-left3)
30.0: (take-formation-slot drone-right)
31.0: (take-formation-slot drone-right2)
32.0: (take-formation-slot drone-right3)
33.0: (establish-v-formation drone-lead source)
34.0: (formation-cruise drone-lead source destination)
35.0: (close-up-wings drone-lead destination)
36.0: (complete-formation-mission drone-lead destination)
Plan-Length:37
```

Five observations worth drawing out.

The staggered launch is unmistakable. Steps 0 to 2 get the leader airborne and climbing; steps 3 to 7 are five `hold-position` actions advancing the clock from 0 to 5; only at step 8 does the first wing drone begin its own pre-flight check. The disjunctive precondition in `takeoff` produced exactly the discipline it was written for.

Each wing drone costs exactly three steps - pre-flight, takeoff, climb - repeated six times for the six wings, filling steps 8 to 25.

The slot-taking order is arbitrary - left2 before left, right before right2. The drones are independent at this stage and the planner has no reason to prefer an order. It does not matter, because `establish-v-formation` requires them all.

The entire cruise is three actions. Steps 34, 35 and 36 - cruise, close up, complete - are the whole journey, because the route is a single direct leg. A run with a marked restriction in the way would show a repeating cruise/close-up pair for each detour leg, which is the `wings-closed` interlock at work.

Only `drone-lead` appears in the last four steps. This is the defining property of the plan: the leader is the only drone the planner ever moves. All six wing tracks are produced afterwards by `formation_wing_routes`, from the single corridor `extract_formation_corridor` reads out of step 34.

6.1 Record to date

Metric	Value
Runs logged	65
Solved	62
Unsolvable / no plan	2
Other	1 (truncated log, 2026-09-15 18:12:54)

Metric	Value
Open code issues	None

The two unsolvable runs were infeasible combinations of source, destination and marked restriction - correct results rather than defects. The truncated run's log stops immediately after grounding, which looks like the application being closed mid-plan rather than a planner failure.

Change on record: 2026-09-16

The neighbour model described in chapter 5 - the `wing-neighbor` facts, the `neighbor-comm-ok` flag and its checks in `establish-v-formation`, `formation-cruise` and `close-up-wings`, and the `detect-neighbor-link-loss` action - was added on this date, mirroring what Grid-Formation already did. Before it, each drone's readiness was judged only on its own link to the ground controller, with no notion of which drones sit next to each other. `expand_formation_drones` gained step 4 at the same time, because cloning the template's rank-1-only pair would otherwise have wired every rank directly to the leader. The change was verified against ENHSP for 3, 5 and 7-drone formations, including through the full retarget-and-expand pipeline.

7. Summary

- **Only the apex is planned.** ENHSP solves one corridor; every wing track is derived from it geometrically afterwards by `formation_wing_routes`.
- **The drone count must be odd and at least 3** - one apex plus a matched pair per rank. `expand_formation_drones` enforces this with a `ValueError`.
- **The V's shape comes from two numbers**: 17.320508 m back and 10 m out per rank, giving exactly 20 m between every adjacent pair, because 17.320508 is 10 times the square root of 3.
- **The domain is generic over the drone count**, because every collective action uses `(forall (?w - drone) (or (not (wing-drone ?w)) ...))` rather than naming drones.
- *`wings-closed` is the interlock* that stops the leader flying two legs without its wings catching up.
- *`close-up-wings` moves the entire wing set in one action*, using a doubly-nested conditional delete - the most advanced PDDL construction in the project.
- *The turn bisector in `formation_member_route` is essential*: the naive before-to-after-the-bend shortcut collapses back to the original bearing and would pinch the wings into the lead track at every detour bend.
- **The no-fly buffer is padded by the formation's lateral reach**, so the whole V clears a marked area rather than just the apex's centreline.
- **Two communication channels**: `communication-ok` to the ground, needed for every action; `neighbor-comm-ok` to the adjacent drones on the same wing, needed for the three collective actions.
- **The neighbour topology is a chain per arm**, not a mesh - each drone talks to the aircraft immediately inboard and outboard of it.
- **At runtime there is still no peer link.** The neighbour model is a planning-time abstraction; cohesion is achieved by each drone flying its own precomputed parallel track.